École Nationale Supérieure d'Arts et Métiers – Meknès
Filière Intelligence Artificielle – 4e année

Cahier des Charges

Sawti-ASR-FineTuner & Data-Augmenter

Pipeline ASR & MLOps Audio pour la Darija marocaine
et le code-switching français–arabe

Réalisé par : Mohamed Amine Belasri
Yahya Amajane

Encadré par : Pr. Hosni
Pr. Menaoui

Année universitaire 2026–2027

Table des matières

1 Présentation du projet **5**
- 1.1 Titre du projet . . . 5
- 1.2 Problème à résoudre . . . 5
- 1.3 Objectif de la solution . . . 5
- 1.4 Périmètre . . . 6
- 1.5 Utilisateurs visés . . . 6

2 Exigences fonctionnelles **7**
- 2.1 Fonctionnalités principales . . . 7
- 2.2 Entrées attendues . . . 7
- 2.3 Sorties attendues . . . 7
- 2.4 Scénarios d'utilisation . . . 8
- 2.5 Priorité des fonctionnalités . . . 8

3 Diagrammes UML **9**
- 3.1 Diagramme de cas d'utilisation . . . 9
- 3.2 Diagramme d'activité . . . 10
- 3.3 Diagramme d'états-transitions . . . 10

4 Données **12**
- 4.1 Type de données . . . 12
- 4.2 Sources des données . . . 12
- 4.3 Description du jeu de données . . . 13
- 4.4 Prétraitement . . . 13

5 Approche IA **14**
- 5.1 Tâche . . . 14
- 5.2 Entrée et sortie du modèle . . . 14
- 5.3 Approche et modèle envisagés . . . 14
- 5.4 Métriques d'évaluation . . . 15
- 5.5 Résultat minimum attendu . . . 16
- 5.6 Plan d'expériences . . . 16

6 Exigences non fonctionnelles **17**
- 6.1 Mode d'utilisation . . . 17
- 6.2 Contraintes matérielles et temps d'exécution . . . 17
- 6.3 Qualité, reproductibilité et traçabilité . . . 17
- 6.4 Environnement technique . . . 17

7 Planning et livrables **19**
7.1 Phases du projet 19
7.2 Échéancier 19
7.3 Livrables 19
7.4 Risques et solutions de repli 19

8 Annexes **21**
8.1 Glossaire 21
8.2 Références et sources des données 21
8.3 Diagrammes complémentaires 22

Liste des tableaux

1.1 Matrice d'inclusion et d'exclusion du projet 6

2.1 Fonctionnalités principales 7
2.2 Priorité des fonctionnalités 8

4.1 Sources des données 12

5.1 Hyperparamètres de départ 15
5.2 Objectifs de performance par rapport à la baseline 16
5.3 Plan d'expériences (ablations) 16
5.4 Format du tableau de résultats (CS : code-switching) 16

6.1 Environnement technique 18

7.1 Échéancier et jalons (S1 à S16) 20
7.2 Risques et solutions de repli 20

Table des figures

3.1 Diagramme de cas d'utilisation 9
3.2 Diagramme d'activité du pipeline 10
3.3 Diagramme d'états-transitions d'une version de modèle 11

5.1 Architecture globale du pipeline 15

Chapitre 1

Présentation du projet

1.1 Titre du projet

Sawti-ASR-FineTuner & Data-Augmenter – Pipeline ASR & MLOps Audio.

Il s'agit d'un projet d'initiation de 4e année (filière IA, ENSAM Meknès), réalisé en binôme sur un semestre (16 semaines), dans le cadre de la plateforme Sawti (thème : continuité, Arabizi et code-switching).

1.2 Problème à résoudre

Les modèles fondationnels de reconnaissance automatique de la parole (ASR), comme Whisper, sont entraînés principalement sur des langues bien dotées en données. Ils donnent de mauvais résultats sur la Darija marocaine, pour trois raisons :

- **Rareté des données** : il existe peu de corpus de Darija transcrits.
- **Code-switching** : les locuteurs alternent la Darija et le français dans une même phrase.
- **Conditions réelles** : les enregistrements contiennent du bruit de fond (rue, café, téléphone), absent des corpus propres.

De plus, adapter ces modèles par un *full fine-tuning* coûte cher en calcul, alors que le projet ne dispose que de GPU gratuits (Kaggle et Google Colab, T4/P100).

1.3 Objectif de la solution

Le projet vise à concevoir un pipeline MLOps automatisé qui :

1. augmente les données audio (SpecAugment, injection de bruit, variations de pitch et de vitesse) pour compenser leur rareté ;
2. fine-tune efficacement le modèle Whisper avec LoRA (PEFT) ;
3. évalue la robustesse du modèle sur trois jeux de test : propre, bruité et code-switching ;
4. trace toutes les expériences et versionne les modèles avec MLflow.

L'objectif mesurable est de battre la baseline (full fine-tuning sans augmentation ni PEFT) en gain relatif de WER. Les valeurs cibles sont données dans la partie 5.

1.4 Périmètre

Le tableau 1.1 précise ce que le projet fait et ne fait pas.

Inclus dans le projet	**Exclu du projet**
Préparation des données audio au format Hugging Face Datasets (nettoyage, rééchantillonnage à 16 kHz, découpage train/validation/test).	Collecte ou enregistrement de nouveaux audios sur le terrain.
Pipeline d'augmentation acoustique et spectrale avec niveaux de bruit contrôlés (SNR).	Transcription manuelle à grande échelle de nouvelles données.
Baseline : Whisper zero-shot et full fine-tuning (version réduite compatible avec un GPU T4).	Interface utilisateur ou application finale de transcription.
Fine-tuning LoRA/PEFT et étude de ses hyperparamètres (rang, modules ciblés).	Déploiement en production (serveur, API temps réel).
Évaluation WER/CER sur trois jeux de test et calcul du gain relatif.	Entraînement de grands modèles (Whisper medium/large) ou depuis zéro.
Suivi des expériences, registre de modèles (MLflow) et script de réentraînement automatisé.	Synthèse vocale (TTS) et traduction.

Table 1.1 – Matrice d'inclusion et d'exclusion du projet

1.5 Utilisateurs visés

— **Équipe de la plateforme Sawti et encadrants** : consultent les résultats et comparent les modèles.

— **Ingénieurs ML / data scientists** : lancent le pipeline, ajoutent des données et réentraînent les modèles.

— **Communauté open source** (utilisateurs secondaires) : réutilisent le code publié sur GitHub et le modèle publié sur Hugging Face.

Chapitre 2

Exigences fonctionnelles

2.1 Fonctionnalités principales

ID	Fonctionnalité	Description
F1	Préparation des données	Charger les corpus, nettoyer, rééchantillonner à 16 kHz, normaliser les transcriptions, découper en train/validation/test.
F2	Augmentation	Ajouter du bruit (ESC-50, NOISEX) à un SNR contrôlé, faire varier le pitch et la vitesse, appliquer SpecAugment.
F3	Baseline	Évaluer Whisper en zero-shot et réaliser un full fine-tuning sans augmentation.
F4	Fine-tuning LoRA	Entraîner des adaptateurs LoRA sur Whisper, avec ou sans augmentation.
F5	Évaluation	Calculer le WER et le CER sur les jeux propre, bruité et code-switching, puis le gain relatif par rapport à la baseline.
F6	Suivi des expériences	Enregistrer dans MLflow les paramètres, les métriques et les artefacts de chaque expérience.
F7	Registre de modèles	Versionner les modèles et identifier la meilleure version.
F8	Réentraînement automatisé	Relancer toute la chaîne avec une seule commande et un fichier de configuration.
F9	Publication	Publier le meilleur modèle et sa documentation sur Hugging Face.

Table 2.1 – Fonctionnalités principales

2.2 Entrées attendues

— Fichiers audio (WAV ou FLAC, mono, 16 kHz) accompagnés de leur transcription.
— Corpus de bruits (ESC-50, NOISEX) pour l'augmentation.
— Fichier de configuration YAML : modèle, hyperparamètres, niveaux de SNR.
— Checkpoints existants, pour reprendre un entraînement interrompu.

2.3 Sorties attendues

— Modèles fine-tunés et adaptateurs LoRA (checkpoints).

— Transcriptions prédites sur les jeux de test.
— Métriques WER et CER par jeu de test, et tableau comparatif des expériences.
— Runs MLflow (paramètres, courbes, artefacts) et versions de modèles dans le registre.

2.4 Scénarios d'utilisation

Scénario 1 – Expérience complète. L'ingénieur rédige un fichier de configuration et lance le pipeline. Les données sont préparées puis augmentées, le modèle est entraîné avec des checkpoints réguliers, puis évalué sur les trois jeux de test. Les résultats sont enregistrés dans MLflow. Si le modèle est meilleur que la version actuelle, il est ajouté au registre.

Scénario 2 – Réentraînement avec de nouvelles données. De nouvelles données sont ajoutées au corpus. L'ingénieur relance la même commande. Une nouvelle version du modèle est produite et comparée à l'ancienne.

Scénario 3 – Reprise après une coupure du GPU. La session Kaggle ou Colab s'arrête pendant l'entraînement. L'ingénieur relance le pipeline, qui reprend au dernier checkpoint sauvegardé.

Scénario 4 – Comparaison des expériences. L'encadrant ouvre MLflow et compare les runs (ablations : avec ou sans augmentation, LoRA ou full fine-tuning, différents rangs et niveaux de SNR).

2.5 Priorité des fonctionnalités

ID	Fonctionnalité	Priorité
F1	Préparation des données	Indispensable
F2	Augmentation	Indispensable
F3	Baseline	Indispensable
F4	Fine-tuning LoRA	Indispensable
F5	Évaluation	Indispensable
F6	Suivi des expériences	Importante
F7	Registre de modèles	Importante
F8	Réentraînement automatisé	Importante
F9	Publication	Optionnelle

TABLE 2.2 – Priorité des fonctionnalités

Chapitre 3

Diagrammes UML

Le projet étant un pipeline de traitement sans interface graphique, nous retenons les trois diagrammes les plus utiles : le diagramme de cas d'utilisation (qui fait quoi), le diagramme d'activité (déroulement du pipeline) et le diagramme d'états-transitions (cycle de vie d'une version de modèle dans le registre).

3.1 Diagramme de cas d'utilisation

L'ingénieur ML utilise toutes les fonctions du pipeline. L'encadrant consulte et compare les résultats. MLflow est un acteur système qui stocke les expériences et les versions de modèles.

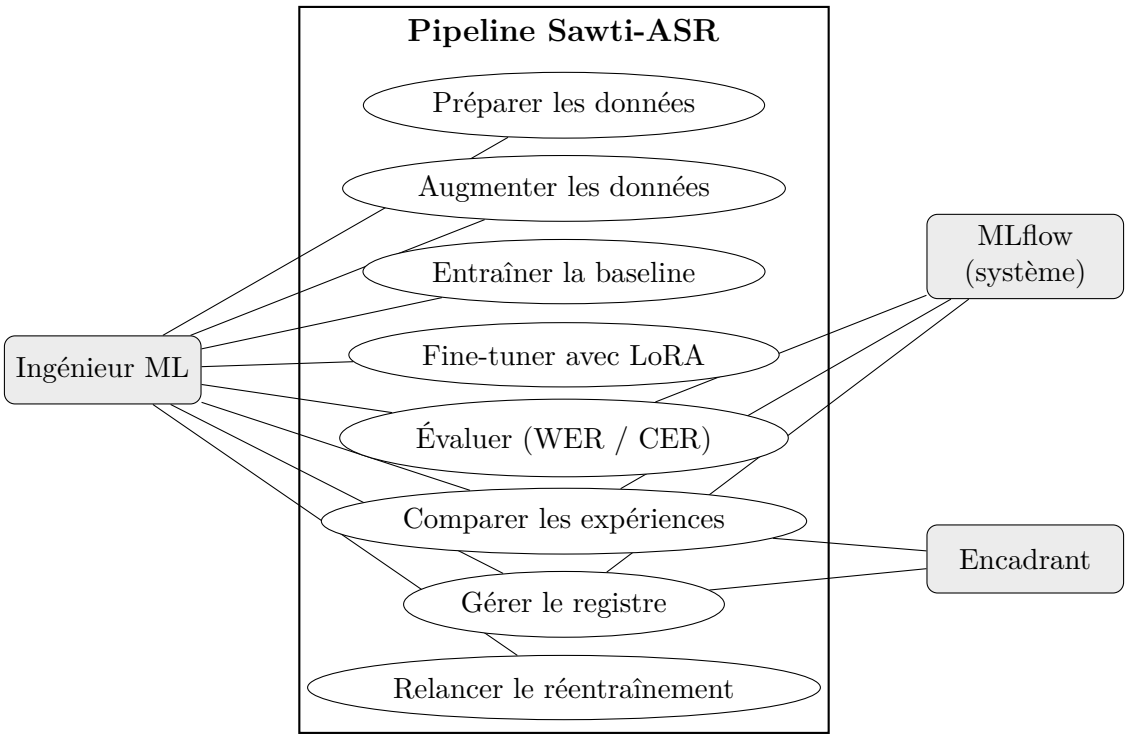


FIGURE 3.1 – Diagramme de cas d'utilisation

3.2 Diagramme d'activité

Le diagramme suivant décrit le déroulement d'une exécution complète du pipeline.

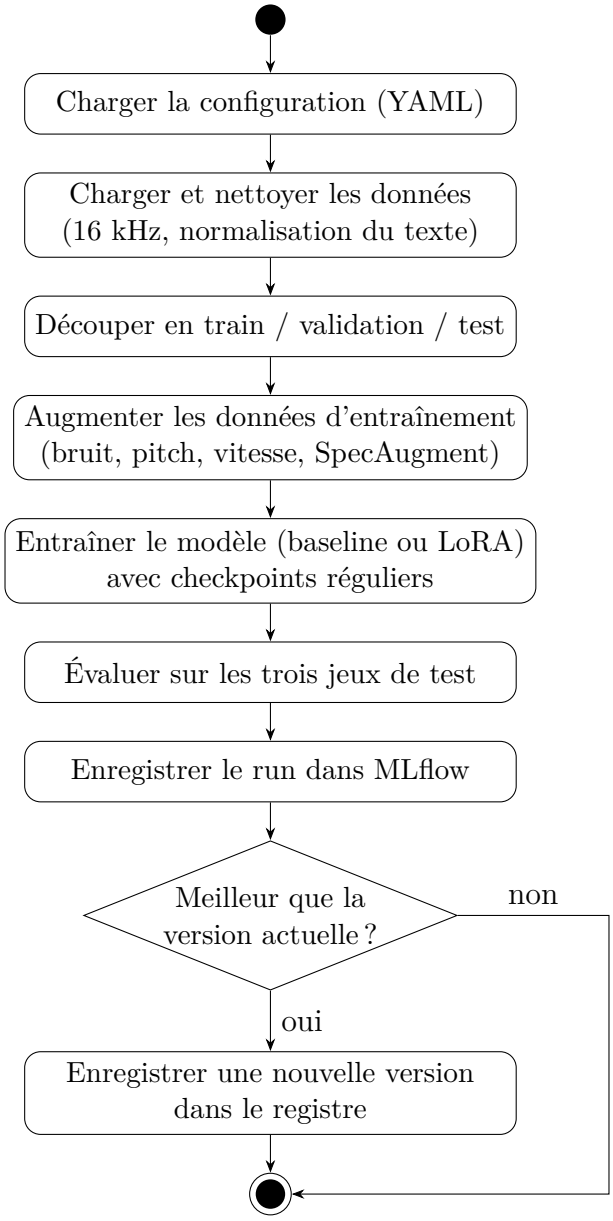


FIGURE 3.2 – Diagramme d'activité du pipeline

3.3 Diagramme d'états-transitions

L'entité qui change d'état est la **version de modèle** : elle est entraînée, évaluée, puis rejetée ou promue, et enfin archivée lorsqu'une meilleure version la remplace. Dans MLflow, ces états seront représentés par des étiquettes ou des alias de version.

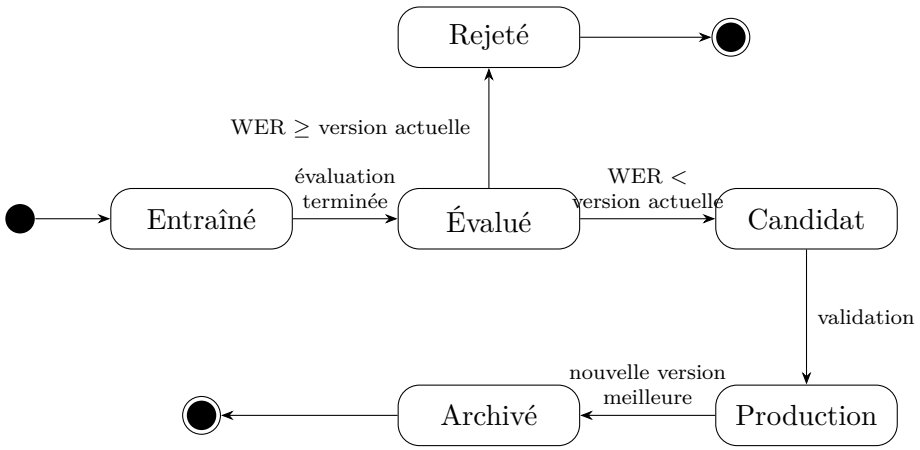


FIGURE 3.3 – Diagramme d'états-transitions d'une version de modèle

Chapitre 4

Données

4.1 Type de données

Le projet manipule trois types de données :

— **Audio** : enregistrements de parole en Darija marocaine, avec ou sans code-switching français–arabe.

— **Texte** : transcription écrite de chaque enregistrement (cible de l'apprentissage).

— **Bruits** : enregistrements de bruits de fond, utilisés pour l'augmentation et pour construire le jeu de test bruité.

4.2 Sources des données

Source	**Rôle**	**Remarques**
Corpus Sawti	Principal	Fourni par les encadrants. Volume et format à confirmer.
Datasets Atlas IA (Hugging Face) : DODa audio, MoulSot	Complément / plan B	Darija marocaine ; MoulSot annonce environ 1 500 h d'audio dont environ 80 h de transcriptions de qualité et du code-switching (chiffres annoncés par les auteurs, à vérifier).
Casablanca	Test code-switching	Corpus multidialectal arabe avec étiquettes de code-switching, dont l'arabe marocain (environ 48 h pour huit dialectes).
ESC-50	Bruits	2 000 extraits de 5 s répartis en 50 classes de sons environnementaux.
NOISEX-92	Bruits	Corpus de bruits de référence pour l'évaluation de la robustesse.

Table 4.1 – Sources des données

Les licences et les conditions d'accès de chaque source seront vérifiées avant utilisation.

4.3 Description du jeu de données

— **Volume** : à confirmer avec les encadrants pour le corpus Sawti. Si le corpus est trop petit, il sera complété par les datasets publics.

— **Variables** : fichier audio, transcription, durée, identifiant du locuteur (si disponible), indicateur de code-switching.

— **Classes** : la tâche n'est pas une classification. Les données sont en revanche réparties en trois jeux de test :

  — **Propre** : enregistrements de test sans modification ;

  — **Bruité** : les mêmes enregistrements mélangés à des bruits à SNR de 20, 10, 5 et 0 dB ;

  — **Code-switching** : enregistrements contenant à la fois de la Darija et du français.

4.4 Prétraitement

1. **Conversion audio** : passage en mono et rééchantillonnage à 16 kHz, la fréquence attendue par Whisper.
2. **Filtrage** : suppression des fichiers vides ou corrompus, et des enregistrements de plus de 30 s (limite de la fenêtre d'entrée de Whisper).
3. **Normalisation du texte** : suppression de la ponctuation, passage en minuscules pour le texte latin, et choix d'une convention d'écriture unique (alphabet arabe ou latin/Arabizi), appliquée à tout le corpus.
4. **Découpage** : 80 % entraînement, 10 % validation, 10 % test, en séparant les locuteurs quand l'information existe, pour éviter qu'un même locuteur se trouve dans l'entraînement et dans le test.
5. **Séparation des bruits** : les fichiers de bruit utilisés pour construire le jeu de test bruité ne sont jamais utilisés pendant l'entraînement.
6. **Format** : les données sont stockées au format Hugging Face Datasets.

Chapitre 5

Approche IA

5.1 Tâche

Il s'agit d'une tâche de **reconnaissance automatique de la parole** (ASR) : transformer un signal audio en texte.

5.2 Entrée et sortie du modèle

— **Entrée** : un signal audio de 30 s au plus, converti en log-spectrogramme de Mel.
— **Sortie** : la transcription textuelle de l'audio.

5.3 Approche et modèle envisagés

Modèle de base. Nous utilisons **Whisper** (OpenAI), un modèle de type Transformer encodeur–décodeur pré-entraîné. Les GPU disponibles (T4 ou P100, 16 Go) limitent le choix aux tailles *tiny* (39 M paramètres), *base* (74 M) et *small* (244 M). Whisper ne possédant pas de code de langue propre à la Darija, le code de langue arabe sera utilisé.

Baseline.

— **Whisper zero-shot** : le modèle sans entraînement, comme point de départ.
— **Full fine-tuning sans augmentation** : tous les paramètres sont entraînés. Pour tenir sur un T4, il est réalisé sur une taille de modèle réduite et, si nécessaire, sur un sous-ensemble des données. La même configuration est utilisée pour la comparaison avec LoRA.

Approche proposée.

— **Augmentation des données** :
  — sur le signal (Audiomentations) : ajout de bruit à SNR contrôlé, variation du pitch (environ ±2 demi-tons) et de la vitesse (facteur 0,9 à 1,1) ;
  — sur le spectrogramme (SpecAugment) : masquage aléatoire de bandes de fréquences et de segments de temps.
— **Fine-tuning LoRA (PEFT)** : le modèle de base est gelé ; seules de petites matrices de rang faible, ajoutées aux couches d'attention, sont entraînées. Cela réduit fortement la mémoire nécessaire et la taille des modèles sauvegardés.

Architecture globale. La figure 5.1 présente l'enchaînement des modules du pipeline.

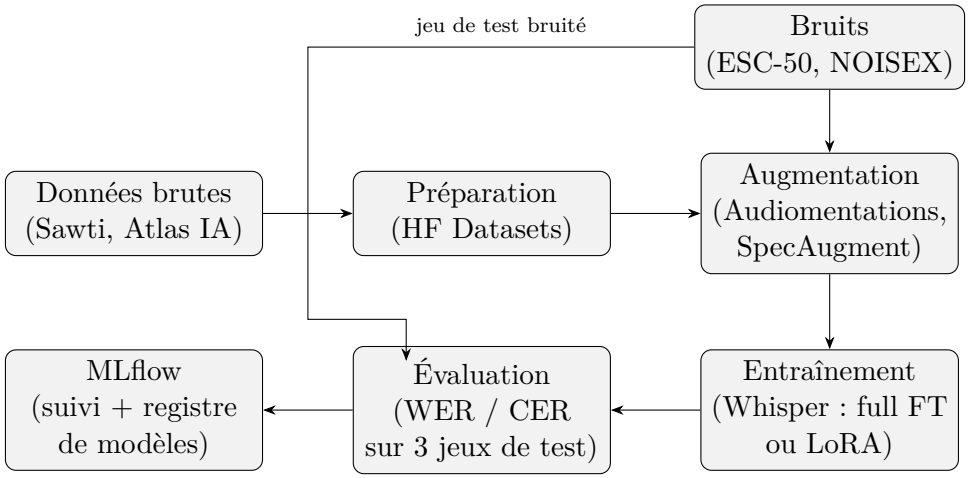


FIGURE 5.1 – Architecture globale du pipeline

Hyperparamètres de départ. Le tableau 5.1 donne les valeurs initiales, qui seront ajustées selon les premiers résultats et le volume réel des données.

Paramètre	**Full fine-tuning**	**LoRA**
Modules ciblés	tous	`q_proj`, `v_proj`
Rang r	–	16 (tests : 8 et 32)
α LoRA	–	32
Dropout LoRA	–	0,05
Taux d'apprentissage	1×10^{-5}	1×10^{-3}
Taille de batch effective	16 (accumulation de gradients)	16
Précision	fp16	fp16
Gradient checkpointing	oui	oui
Sauvegarde	toutes les 200 étapes	toutes les 200 étapes

TABLE 5.1 – Hyperparamètres de départ

5.4 Métriques d'évaluation

— **WER** (Word Error Rate), taux d'erreur sur les mots :

$$\text{WER} = \frac{S + D + I}{N}$$

où S, D et I sont les nombres de substitutions, suppressions et insertions de mots, et N le nombre de mots de la référence.

— **CER** (Character Error Rate) : même calcul au niveau des caractères, utile pour la Darija dont l'orthographe varie.

— **Gain relatif de WER** par rapport à la baseline :

$$\text{Gain} = \frac{\text{WER}_{\text{baseline}} - \text{WER}_{\text{modèle}}}{\text{WER}_{\text{baseline}}}$$

Ces métriques sont calculées séparément sur les jeux propre, bruité (par niveau de SNR) et code-switching.

5.5 Résultat minimum attendu

Jeu de test	**Objectif minimum**	**Objectif visé**
Bruité	gain relatif $\geq 10\,\%$	$\geq 20\,\%$
Code-switching	gain relatif $\geq 5\,\%$	$\geq 10\,\%$
Propre	perte ≤ 1 point de WER	égal ou meilleur

Table 5.2 – Objectifs de performance par rapport à la baseline

Ces objectifs seront ajustés après la mesure de la baseline.

5.6 Plan d'expériences

Expérience	**Description**
E0	Whisper zero-shot (sans entraînement)
E1	Full fine-tuning sans augmentation (**baseline**)
E2	Full fine-tuning avec augmentation
E3	LoRA sans augmentation
E4	LoRA avec augmentation, rangs $r = 8$, 16 et 32
E5	Meilleure configuration de E4, avec différents niveaux de SNR d'entraînement

Table 5.3 – Plan d'expériences (ablations)

Les résultats seront présentés sous la forme suivante :

Exp.	**Param. entraînés**	**WER propre**	**WER bruité**	**WER CS**	**CER**	**Gain bruité**
E0	–					
E1						réf.
...						

Table 5.4 – Format du tableau de résultats (CS : code-switching)

Chapitre 6

Exigences non fonctionnelles

6.1 Mode d'utilisation

Le pipeline est utilisé sans interface graphique : il est lancé par des scripts en ligne de commande ou depuis des notebooks Kaggle et Google Colab. Chaque exécution est entièrement décrite par un fichier de configuration YAML. Les résultats sont consultés dans l'interface web de MLflow.

6.2 Contraintes matérielles et temps d'exécution

— **GPU gratuits uniquement** (Kaggle, Google Colab : T4 ou P100, 16 Go). Les quotas d'utilisation et la durée maximale des sessions sont limités ; leurs valeurs exactes seront vérifiées au début du projet.

— **Reprise après coupure** : un checkpoint est sauvegardé régulièrement et l'entraînement peut reprendre au dernier checkpoint.

— **Économie de mémoire** : précision fp16, gradient checkpointing et accumulation de gradients.

— **Temps de réponse** : non critique, car il n'y a pas d'utilisation en temps réel. La durée d'entraînement et d'évaluation de chaque expérience est tout de même enregistrée.

6.3 Qualité, reproductibilité et traçabilité

— Graines aléatoires fixées et versions des bibliothèques figées (`requirements.txt`).

— Chaque expérience est enregistrée dans MLflow avec sa configuration, ses métriques et ses artefacts.

— Code organisé en modules, documenté (README) et versionné sur GitHub.

6.4 Environnement technique

Élément	**Choix**
Langage	Python 3
Apprentissage profond	PyTorch, Hugging Face Transformers
Fine-tuning efficace	Hugging Face PEFT (LoRA)
Données	Hugging Face Datasets
Audio	Librosa, Audiomentations
Évaluation	bibliothèque de calcul du WER/CER (par exemple jiwer ou evaluate)
Suivi et registre	MLflow
Calcul	Kaggle, Google Colab
Versionnement et publication	Git, GitHub, Hugging Face Hub

TABLE 6.1 – Environnement technique

Chapitre 7

Planning et livrables

7.1 Phases du projet

1. **Cadrage** : étude du sujet, état de l'art, cahier des charges.
2. **Données** : collecte, prétraitement, jeux de test.
3. **Baseline** : Whisper zero-shot et full fine-tuning.
4. **Augmentation et LoRA** : développement des deux modules.
5. **Expériences** : ablations et analyse des résultats.
6. **Industrialisation** : automatisation, registre de modèles, documentation.
7. **Clôture** : rapport final et soutenance.

7.2 Échéancier

La personne A est responsable des données, de l'augmentation et de l'évaluation. La personne B est responsable du fine-tuning, de la baseline, de MLflow et de l'automatisation.

7.3 Livrables

— Cahier des charges (S2).

— Jeux de données préparés et documentés (propre, bruité, code-switching).

— Code du pipeline : préparation, augmentation, entraînement, évaluation, réentraînement automatisé (dépôt GitHub).

— Modèles entraînés enregistrés dans le registre MLflow.

— Résultats d'évaluation : tableau comparatif des expériences.

— Rapport final et support de soutenance.

— (Optionnel) Meilleur modèle publié sur Hugging Face.

7.4 Risques et solutions de repli

Semaines	Personne A	Personne B	Jalon
S1–S2	Étude du sujet, état de l'art, cahier des charges		Cahier des charges
S3–S4	Collecte et exploration des données	Environnement, dépôt GitHub, MLflow, Whisper zero-shot	
S5–S6	Prétraitement, découpage, trois jeux de test	Baseline full fine-tuning	Baseline mesurée
S7–S8	Pipeline d'augmentation (bruit, pitch, vitesse, SpecAugment)	Premiers entraînements LoRA	
S9–S10	Scripts d'évaluation complets (WER/CER, gain relatif)	LoRA avec augmentation	Point de mi-parcours
S11–S12	Ablations sur les niveaux de SNR	Ablations sur les rangs LoRA, registre de modèles	Résultats des expériences
S13	Analyse de la robustesse	Automatisation du réentraînement	Pipeline complet
S14	Documentation, nettoyage du code, publication		Dépôt final
S15	Rédaction du rapport final		Rapport final
S16	Préparation et soutenance		Soutenance

Table 7.1 – Échéancier et jalons (S1 à S16)

Risque	Solution de repli
Corpus Sawti indisponible ou trop petit	Utiliser les datasets publics (Atlas IA, Casablanca)
Coupures ou quotas des GPU gratuits	Checkpoints réguliers, reprise automatique, répartition des runs sur Kaggle et Colab
Full fine-tuning trop lourd pour un T4	Modèle plus petit (tiny ou base) et sous-ensemble des données, avec la même configuration pour LoRA
Transcriptions incohérentes (alphabet arabe ou latin)	Normalisation du texte et suivi du CER en plus du WER
Pas de gain mesurable	Analyse par type de bruit et par niveau de SNR, ajustement des hyperparamètres

Table 7.2 – Risques et solutions de repli

Chapitre 8

Annexes

8.1 Glossaire

ASR
Automatic Speech Recognition : reconnaissance automatique de la parole.

Arabizi
Écriture de l'arabe ou de la Darija avec l'alphabet latin et des chiffres.

Code-switching
Alternance de deux langues dans une même phrase (ici Darija et français).

WER / CER
Taux d'erreur sur les mots / sur les caractères.

SNR
Signal-to-Noise Ratio : rapport entre le niveau de la parole et celui du bruit, en décibels.

SpecAugment
Augmentation qui masque des bandes de fréquences et des segments de temps du spectrogramme.

Fine-tuning
Réentraînement d'un modèle pré-entraîné sur de nouvelles données.

Zero-shot
Utilisation d'un modèle sans entraînement supplémentaire.

LoRA / PEFT
Low-Rank Adaptation / Parameter-Efficient Fine-Tuning : fine-tuning qui n'entraîne qu'une petite partie des paramètres.

MLOps
Pratiques d'automatisation et de suivi du cycle de vie des modèles.

Checkpoint
Sauvegarde de l'état d'un entraînement pour pouvoir le reprendre.

8.2 Références et sources des données

— A. Radford et al., *Robust Speech Recognition via Large-Scale Weak Supervision* (Whisper), 2022.

— E. Hu et al., *LoRA : Low-Rank Adaptation of Large Language Models*, 2021.
— D. Park et al., *SpecAugment : A Simple Data Augmentation Method for Automatic Speech Recognition*, 2019.
— K. Piczak, *ESC : Dataset for Environmental Sound Classification*, 2015.
— A. Varga et H. Steeneken, NOISEX-92, 1993.
— Corpus Casablanca : `https://arxiv.org/abs/2410.04527`
— Collection Atlas IA (Hugging Face) : `https://huggingface.co/collections/atlasia/speech-data-and-models`
— Modèle MoulSot : `https://huggingface.co/01Yassine/moulsot.v0.3`

8.3 Diagrammes complémentaires

Les diagrammes de séquence et de classes n'ont pas été retenus dans ce document, car ils apportent peu d'information supplémentaire pour un pipeline sans interface. Ils pourront être ajoutés au rapport final si nécessaire.